HDOC Day-53

Product of Array Except Self Using Pointers in C

Problem Statement

Read a variable-length integer array nums[]. Pass its address to product_except_self() and create answer[] where each element equals the product of all other elements. Do not use division. Print the results separated by commas.

Algorithm

1. Read a positive array size and reject invalid sizes.

2. Declare a variable-length integer array nums[size] and read its elements.

3. Pass nums (its first-element address) and size to product_except_self().

4. Declare answer[size] and initialize prefix = 1.

5. Scan from left to right: store the current prefix in answer[i], then multiply prefix by nums[i].

6. Initialize suffix = 1 and scan from right to left: multiply answer[i] by suffix, then multiply suffix by nums[i].

7. Display answer[] in original order, separated by commas; do not use division.

8. Stop. Assume the required intermediate products fit in a 32-bit signed integer as specified.

Test Case Table

TC	Size	Elements	Expected output	Purpose
1	4	1 2 3 4	24, 12, 8, 6	Professor example 1
2	5	-1 1 0 -3 3	0, 0, 9, 0, 0	Professor example 2; one zero
3	3	0 0 2	0, 0, 0	Two zeros
4	3	2 3 4	12, 8, 6	All positive
5	3	-1 -2 -3	6, 3, 2	Negative elements
6	2	7 8	8, 7	Two elements
7	1	9	1	One element: empty product = 1
8	0	Not applicable	Invalid size. Enter an integer from 1 to 1000.	Invalid size

C Program

```
#include <stdio.h>

void product_except_self(const int *nums, int size)
{
    int answer[size];
    int i;
    int prefix = 1, suffix = 1;
```

```
    for (i = 0; i < size; i++) {
        answer[i] = prefix;
        prefix *= *(nums + i);
    }
    for (i = size - 1; i >= 0; i--) {
        answer[i] *= suffix;
        suffix *= *(nums + i);
    }
    printf("Product except self: ");
    for (i = 0; i < size; i++) {
        if (i > 0) printf(", ");
        printf("%d", answer[i]);
    }
    printf("\n");
}

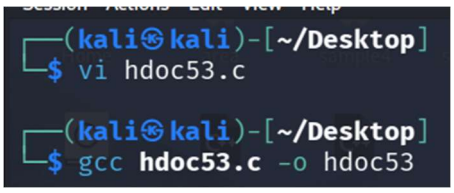
int main(void)
{
    int size, i;
    printf("Enter array size: ");
    if (scanf("%d", &size) != 1 || size <= 0 || size > 1000) {
        printf("Invalid size. Enter an integer from 1 to 1000.\n");
        return 1;
    }
    int nums[size];
    printf("Enter %d array elements:\n", size);
    for (i = 0; i < size; i++) {
        if (scanf("%d", nums + i) != 1) {
            printf("Invalid array element.\n");
            return 1;
        }
    }
    product_except_self(nums, size);
    return 0;
}
```

Compilation and Execution

gcc hdoc53.c -o hdoc53

./hdoc53

Compilation and Execution



Test Case 1

Input: size = 4; elements = 1 2 3 4. Expected: 24, 12, 8, 6.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc53
Enter array size: 4
Enter 4 array elements:
1 2 3 4
Product except self: 24, 12, 8, 6
```

Test Case 2

Input: size = 5; elements = -1 1 0 -3 3. Expected: 0, 0, 9, 0, 0.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc53
Enter array size: 5
Enter 5 array elements:
-1 1 0 -3 3
Product except self: 0, 0, 9, 0, 0
```

Test Case 3

Input: size = 3; elements = 0 0 2. Expected: 0, 0, 0.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc53
Enter array size: 5
Enter 5 array elements:
-1 1 0 -3 3
Product except self: 0, 0, 9, 0, 0
```

Test Case 4

Input: size = 3; elements = 2 3 4. Expected: 12, 8, 6.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc53
Enter array size: 3
Enter 3 array elements:
2 3 4
Product except self: 12, 8, 6
```

Test Case 5

Input: size = 3; elements = -1 -2 -3. Expected: 6, 3, 2.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc53
Enter array size: 3
Enter 3 array elements:
-1 -2 -3
Product except self: 6, 3, 2
```

Test Case 6

Input: size = 2; elements = 7 8. Expected: 8, 7.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc53
Enter array size: 2
Enter 2 array elements:
7 8
Product except self: 8, 7
```

Test Case 7

Input: size = 1; elements = 9. Expected: 1.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc53
Enter array size: 1
Enter 1 array elements:
9
Product except self: 1
```

Test Case 8

Input: size = 0; elements = Not applicable. Expected: Invalid size. Enter an integer from 1 to 1000..

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc53
Enter array size: 0
Invalid size. Enter an integer from 1 to 1000.
```